

Exercise 2: Mean-Shift Tracking

Dino Džaferagić

I. INTRODUCTION

This project has two core tasks. The first task is mean-shift mode seeking on response maps. The second task is a color-based mean-shift tracker. I implemented both tasks in Python and tested them on the required data. I then ran extra and bonus experiments from the grading section.

II. EXPERIMENTS

This section follows the grading tasks.

A. Mean-shift mode seeking

I tested two kernels (Epanechnikov, Gaussian), three bandwidths (11, 21, 31), three stop criteria (1.0, 0.5, 0.1), and four start points. Table I shows the main pattern from the runs. Figure 1 shows convergence trajectories on the provided response map.

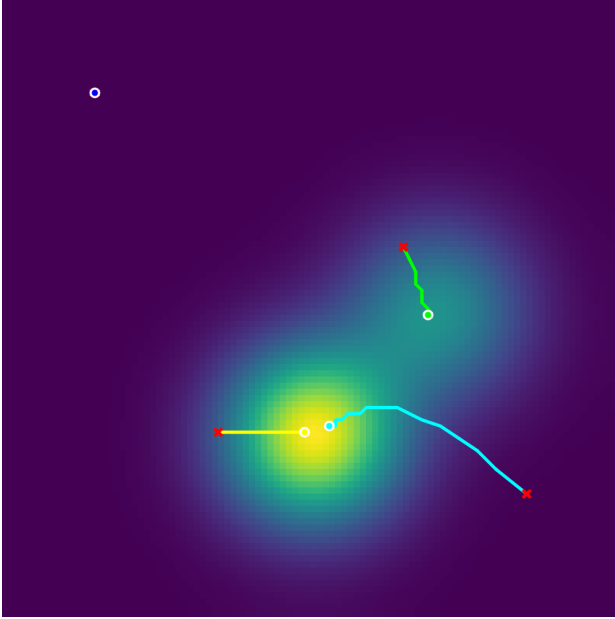


Figure 1. Mean-shift convergence trajectories on the provided response map. Red marks show start points. Colored lines show iterative paths.

Table I
MODE-SEEKING BEHAVIOR SUMMARY

Kernel	Bandwidth	Stop eps	Typical result
Epanechnikov	11	1.0	fast, local moves
Epanechnikov	21	0.5	stable tradeoff
Epanechnikov	31	0.1	often hits 100 steps
Gaussian	11	0.5	moderate speed
Gaussian	21	0.1	many long runs
Gaussian	31	0.5	stable global pull

The start point has strong effect. When the start is far from a strong peak, the method can stall or move slowly. A strict stop threshold (0.1) often increases steps a lot. In my setup, bandwidth 21 with stop threshold 0.5 gave a good balance.

B. Tracker on five sequences

I evaluated the tracker on five VOT2014 sequences: **polarbear**, **hand1**, **fernando**, **drunk**, **basketball**. I count a tracking failure when overlap is 0 or less; after each failure, the evaluation skips 5 frames and then reinitializes the tracker. Table II gives the required results.

Table II
REQUIRED TRACKER RESULTS (5 SEQUENCES)

Sequence	FPS	Failures
polarbear	381.0	0
hand1	289.6	4
fernando	87.1	1
drunk	169.7	0
basketball	224.2	1
Total / Avg	230.3	6

C. Own function for mode seeking

I added a custom response landscape with three peaks in `run_mode_seeking.py`. The method converged on this map and showed the same behavior trends as on the provided map. This confirms that the implementation is not tied to one specific response shape. From the script output, the provided map had `avg steps=23.00`, `min=0`, `max=100`, while the custom map had `avg steps=8.26`, `min=3`, `max=29`. This shows that the custom landscape was easier for convergence than the provided one.

D. Failure case analysis

The hardest case was **hand1** with 4 failures. In good frames, the box is centered on the raised hand. In bad frames, fast hand motion and blur push the box toward wrist or nearby background.

In **drunk**, the target is a dark car in a street scene with other similar cars. Good frames show the box on the target car. Bad frames are suspicious when the box shifts to another car region with similar color and shape.

In **fernando**, the scene contains two cats in an indoor environment. Good frames lock on the intended cat. Suspicious frames appear when the cats cross and the box may switch identity for a short time.

In **basketball**, the target player in green is tracked well in many frames. In bad frames, the box can move to another nearby player with similar jersey color.

In **polarbear**, the tracker is stable over the full sequence (0 failures). Some frames are still suspicious because white target regions are similar to snow, but no official failure occurred.

Main improvement idea: combine color cues with an extra feature (for example gradient or texture) and use stronger motion prediction during fast changes.

E. Parameter impact

I tested bins, kernel size, stop criterion / iterations, and update speed. Table III summarizes total failures and average FPS on the same five sequences. For backprojection weights, I used $v = q/(p + \epsilon)$ (without square root). The slide form

Table III
PARAMETER IMPACT SUMMARY

Setting	Total failures	Avg FPS
<code>nbins=8/16/32</code>	9/6/10	408.1/255.3/107.2
<code>window_scale=1.5/2.0/2.5</code>	6/6/6	208.5/295.2/259.3
<code>kernel_sigma=0.8/1.0/1.2</code>	27/6/8	104.4/275.1/502.5
<code>stop_eps=1.0/0.5/0.2</code>	9/6/8	453.4/278.7/122.5
<code>max_iters=10/20/35</code>	7/6/6	348.8/285.9/219.7
<code>model_alpha=0.0/0.05/0.2</code>	5/6/7	294.9/285.7/274.0

with $\sqrt{q/(p+\epsilon)}$ is also valid, but this simpler ratio gave stable behavior in this implementation and parameter setup.

High bin count increased cost and did not help in this setup. Kernel settings mattered a lot; poor kernel scale (`kernel_sigma=0.8`) gave many failures. Very loose stopping (`stop_eps=1.0`) was fast but less robust. In this subset, no model update (`model_alpha=0.0`) gave the lowest failure count.

F. Color spaces and background model

I tested four color spaces. I also added background histogram modeling. Table IV shows the result.

Table IV
BONUS EXPERIMENTS

Experiment	Total failures	Avg FPS
BGR	6	276.1
HSV	10	159.2
Lab	7	233.5
YCrCb	9	258.3
no background model	6	262.9
with background model	5	353.5

In this test, BGR gave the best color-space result. Background modeling improved both robustness and speed on this subset.

III. CONCLUSION

I implemented all required parts and completed extra and bonus tasks from the grading section. On the VOT2014 subset, the tracker reached 6 failures in total on five sequences. The main failure source was fast local motion and target ambiguity. Parameter choice had strong impact on both speed and failures. The most useful bonus change in this run was background modeling.